

Introducción a MongoDB

Base de Datos NoSQL Orientada a Documentos

0. Índice

1. ¿Qué es una Base de Datos?
2. ¿Qué es MongoDB?
3. Estructura de Datos en MongoDB
4. Instalación en Ubuntu
5. Acceso y Operaciones Básicas
6. Consultas Avanzadas
7. Conexión con Aplicaciones
8. Casos de Uso y Ventajas

1. ¿Qué es una Base de Datos?

Una base de datos es un sistema organizado para almacenar, gestionar y recuperar información de manera eficiente.

Tipos principales:

- **Relacionales (SQL):** MySQL, PostgreSQL, Oracle
- **NoSQL:** MongoDB, Cassandra, Redis

Características clave:

- Persistencia de datos
- Acceso concurrente
- Integridad y seguridad
- Eficiencia en consultas

2. ¿Qué es MongoDB?

MongoDB es una base de datos NoSQL orientada a documentos, de código abierto y multiplataforma.

Características principales:

- **Alta escalabilidad** horizontal
- **Flexible schema** (esquema flexible)
- **Índices** para optimización

Ventajas:

- Fácil de aprender para desarrolladores web
- Excelente para datos no estructurados
- Alta performance en lectura/escritura

3. Estructura de Datos en MongoDB

Jerarquía de datos:

```
Database (Base de Datos)
├─ Collection (Colección)
│   └─ Document (Documento)
│       └─ Fields (Campos)
```

Ejemplo de documento:

```
{
  "_id": ObjectId("507f1f77bcf86cd799439011"),
  "nombre": "Juan Pérez",
  "email": "juan@example.com",
  "edad": 30,
  "direccion": {
    "calle": "Calle Principal",
    "numero": 123,
    "ciudad": "Madrid"
  },
  "hobbies": ["lectura", "ciclismo", "programación"]
}
```

4. Instalación de MongoDB en Ubuntu

Revisad siempre primero la documentación oficial en: [documentación](#).

Paso 1: Instalar dependencias e importar clave pública

```
sudo apt-get install gnupg curl
curl -fsSL https://www.mongodb.org/static/pgp/server-7.0.asc | \
  sudo gpg -o /usr/share/keyrings/mongodb-server-7.0.gpg \
  --dearmor
```

Paso 2: Crear lista de fuentes

```
echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-7.0.gpg ] https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/7.0 multiverse" \
| sudo tee /etc/apt/sources.list.d/mongodb-org-7.0.list
```

Paso 3: Actualizar e instalar

```
sudo apt-get update
sudo apt-get install -y mongodb-org
```

5. Verificación de Instalación

Paso 4: Iniciar servicio y verificar

```
sudo systemctl start mongod
sudo systemctl enable mongod
sudo systemctl status mongod

# Verificar versión
mongod --version

# Verificar que puedes acceder a la shell
mongosh
```

5. Verificación de Instalación

Dentro de la shell podemos usar una serie de comandos, como:

Comandos básicos de verificación:

```
// Ver bases de datos existentes  
show dbs  
  
// Ver versión actual  
db.version()  
  
// Ver estadísticas  
db.stats()
```

6. Acceso y Operaciones Básicas

Crear/Seleccionar base de datos:

```
use miBaseDatos
```

Crear colección:

```
db.createCollection("usuarios")
```

Ver colecciones:

```
show collections
```

7. Inserciones de Datos

Insertar un documento:

```
db.usuarios.insertOne({  
  nombre: "Ana García",  
  email: "ana@example.com",  
  edad: 25  
})
```

Insertar múltiples documentos:

```
db.usuarios.insertMany([  
  { nombre: "Carlos", email: "carlos@example.com", edad: 28 },  
  { nombre: "María", email: "maria@example.com", edad: 32 },  
  { nombre: "Pedro", email: "pedro@example.com", edad: 29 }  
])
```

8. Consultas Básicas

Buscar todos los documentos:

```
db.usuarios.find()
```

Buscar con formato legible:

```
db.usuarios.find().pretty()
```

Buscar por campo específico:

```
db.usuarios.find({ nombre: "Ana García" })
```

Buscar con múltiples condiciones:

```
db.usuarios.find({ edad: { $gte: 25 }, nombre: "Carlos" })
```

8. Consultas Básicas

Contar documentos:

```
db.usuarios.countDocuments()  
db.usuarios.countDocuments({ edad: { $gte: 30 } })
```

9. Operadores de Consulta

Operadores de comparación:

```
// Mayor que
db.usuarios.find({ edad: { $gt: 25 } })

// Menor o igual que
db.usuarios.find({ edad: { $lte: 30 } })

// En un rango
db.usuarios.find({ edad: { $gte: 25, $lte: 35 } })

// No igual
db.usuarios.find({ edad: { $ne: 30 } })
```

9. Operadores de Consulta

Operadores lógicos:

```
// AND (implícito)
db.usuarios.find({ edad: { $gte: 25 }, nombre: "Carlos" })

// OR
db.usuarios.find({ $or: [{ edad: 25 }, { nombre: "María" }] })
```

10. Actualizaciones y Borrados

Actualizar un documento:

```
// Actualizar primer documento que coincida
db.usuarios.updateOne(
  { nombre: "Ana García" },
  { $set: { edad: 26 } }
)

// Actualizar todos los documentos que coincidan
db.usuarios.updateMany(
  { edad: { $lt: 30 } },
  { $set: { categoria: "joven" } }
)
```

10. Actualizaciones y Borrados

Borrar documentos:

```
// Borrar primer documento que coincida
db.usuarios.deleteOne({ nombre: "Pedro" })

// Borrar todos los documentos que coincidan
db.usuarios.deleteMany({ edad: { $lt: 25 } })

// Borrar toda la colección
db.usuarios.deleteMany({})
```

11. Índices en MongoDB

Crear índice:

```
// Índice ascendente
db.usuarios.createIndex({ email: 1 })

// Índice descendente
db.usuarios.createIndex({ edad: -1 })

// Índice compuesto
db.usuarios.createIndex({ nombre: 1, edad: -1 })

// Índice único
db.usuarios.createIndex({ email: 1 }, { unique: true })
```

11. Índices en MongoDB

Ver índices:

```
db.usuarios.getIndexes()
```

Eliminar índice:

```
db.usuarios.dropIndex({ email: 1 })
```

12. Agregaciones (Consultas Avanzadas)

Pipeline de agregación:

```
db.usuarios.aggregate([
  { $match: { edad: { $gte: 25 } } },
  { $group: { _id: "$categoria", total: { $sum: 1 } } },
  { $sort: { total: -1 } }
])
```

12. Agregaciones (Consultas Avanzadas)

Operadores comunes:

- `$match`: Filtrar documentos
- `$group`: Agrupar documentos
- `$project`: Seleccionar campos
- `$sort`: Ordenar resultados
- `$limit`: Limitar resultados
- `$lookup`: Unir colecciones (JOIN)

13. Conexión con Aplicaciones Node.js

Instalar driver de MongoDB:

```
npm install mongodb
```

13. Conexión con Aplicaciones Node.js

```
const { MongoClient } = require('mongodb');

const uri = "mongodb://localhost:27017";
const client = new MongoClient(uri);

async function run() {
  try {
    await client.connect();
    const database = client.db('miBaseDatos');
    const usuarios = database.collection('usuarios');

    // Insertar documento
    const result = await usuarios.insertOne({
      nombre: "Nuevo Usuario",
      email: "nuevo@example.com"
    });

    console.log(`Documento insertado con ID: ${result.insertedId}`);
  } finally {
    await client.close();
  }
}

run().catch(console.dir);
```

14. Seguridad en MongoDB

Autenticación:

```
# Crear usuario administrador
use admin
db.createUser({
  user: "admin",
  pwd: "password123",
  roles: [ { role: "userAdminAnyDatabase", db: "admin" } ]
})

# Habilitar autenticación en mongod.conf
security:
  authorization: enabled
```

15. Seguridad en MongoDB

Conexión con autenticación:

```
mongosh -u admin -p password123 --authenticationDatabase admin
```

Permisos comunes:

- `read`: Solo lectura
- `readWrite`: Lectura y escritura
- `dbAdmin`: Administración de base de datos
- `userAdmin`: Administración de usuarios

16. Backup y Restore

Crear backup:

```
mongodump --db miBaseDatos --out /ruta/backup/
```

Restaurar backup:

```
mongorestore --db miBaseDatos /ruta/backup/miBaseDatos/
```

Backup específico de colección:

```
mongodump --db miBaseDatos --collection usuarios --out /ruta/backup/
```

16. Backup y Restore

Exportar a JSON:

```
mongoexport --db miBaseDatos --collection usuarios --out usuarios.json
```

Importar desde JSON:

```
mongoimport --db miBaseDatos --collection usuarios --file usuarios.json
```

17. Casos de Uso Ideales para MongoDB

Recomendado para:

- Aplicaciones web y móviles modernas
- Contenido y gestión de medios
- IoT y datos en tiempo real
- Catálogos de productos
- Perfiles de usuarios
- Registros de actividad (logging)
- Datos no estructurados o semi-estructurados

17. Casos de Uso Ideales para MongoDB

No recomendado para:

- Sistemas que requieren transacciones ACID complejas
- Aplicaciones financieras con alta consistencia
- Sistemas legacy con esquemas rígidos
- Aplicaciones con relaciones complejas entre datos

18. Comparación MongoDB vs SQL

Característica	MongoDB	SQL (MySQL/PostgreSQL)
Modelo de datos	Documentos JSON	Tablas con filas/columnas
Esquema	Flexible (schema-less)	Estructurado (schema)
Lenguaje de consulta	MongoDB Query Language	SQL
Escalabilidad	Horizontal (sharding)	Vertical principalmente
Transacciones	Multi-document (4.0+)	ACID completo
Joins	\$lookup (agregación)	JOIN nativo
Índices	Únicos, compuestos, geospaciales	Primarios, únicos, foráneos

19. Buenas Prácticas

1. Diseño de esquema:

- Normalizar vs denormalizar según caso de uso
- Considerar patrones de acceso a datos
- Usar referencias o embedding apropiadamente

2. Índices:

- Crear índices para campos frecuentemente consultados
- Evitar demasiados índices (afecta escritura)
- Usar índices compuestos para consultas múltiples

19. Buenas Prácticas

3. Seguridad:

- Siempre habilitar autenticación
- Usar roles específicos para usuarios
- Mantener MongoDB actualizado

4. Performance:

- Usar proyecciones para limitar campos retornados
- Implementar paginación para grandes conjuntos
- Monitorizar con herramientas como MongoDB Compass

20. Herramientas Útiles

MongoDB Compass:

- GUI oficial para MongoDB
- Visualización de datos
- Construcción de consultas
- Análisis de performance

MongoDB Atlas:

- Base de datos como servicio (DBaaS)
- Gestión automática
- Escalado automático
- Backups automáticos

20. Qué tenéis que hacer

Pasos para tener la nueva versión:

1. Instalar mongodb en la máquina EC2.
2. Probad que la instalación funciona.
3. Adaptar la aplicación en AI Studio para que guarde todos los datos en mongodb.
4. Llevar el nuevo código al repositorio de Github.
5. Dentro de la EC2, en la carpeta del proyecto, ejecutar `git pull`
6. Instalar las nuevas dependencias con `npm install`
7. Compilar de nuevo el proyecto con `npm run build`
8. Desplegar de nuevo el proyecto con el comando `pm2 ...`